



AI Security Scanner for Python



Ahmed Tetteh

```
import google.generativeai as genai
The line 'password = "admin123"' itself, while very simple, introduces several critical security vulnerabilities and demonstrates poor security practices. Here's a breakdown:

## Critical Security Issues:

1. Hardcoded Password:
  * Issue: The password 'admin123' is directly written into the source code.
  * Explanation: This is perhaps the most significant flaw. Any individual with access to the source code (developers, administrators, version control systems, deployment artifacts, accidental leaks) will immediately know the plaintext password.
  * Impact:
    * Insider Threat: Malicious insiders can easily exploit this.
    * Compromise on Code Exposure: If the code repository becomes public or is breached, the password is leaked.
    * Lack of Rotation: Changing the password requires modifying the code and redeploying, making rotation cumbersome and often neglected.
```



Introducing Today's Project!

In this project, I'm going to build an AI security scanner for Python. This will help me learn how development teams catch vulnerabilities in their application before it reaches production. I'm interested in this because it saves a lot of effort on the part of Developers when using manual tools in detecting these vulnerabilities.

Key tools and concepts

Tools I used were Gemini, Gemini API key, Security vulnerabilities and Cursor. Key concepts I learnt include: How to use Gemini API to analyze code for security vulnerabilities, Adding colored terminal output with colorama for severity ratings, Common security vulnerabilities: SQL injection, hardcoded secrets, and weak cryptography. The most important skill was how to implement AI in testing for vulnerability severities before pushing the application to production.

Challenges and wins

This project took me approximately 2 hours. The most challenging part was learning how implementing severity ratings to my responses. It was most rewarding to implement AI in the vulnerability process, eliminating a lot of manual steps, and reducing detection time.

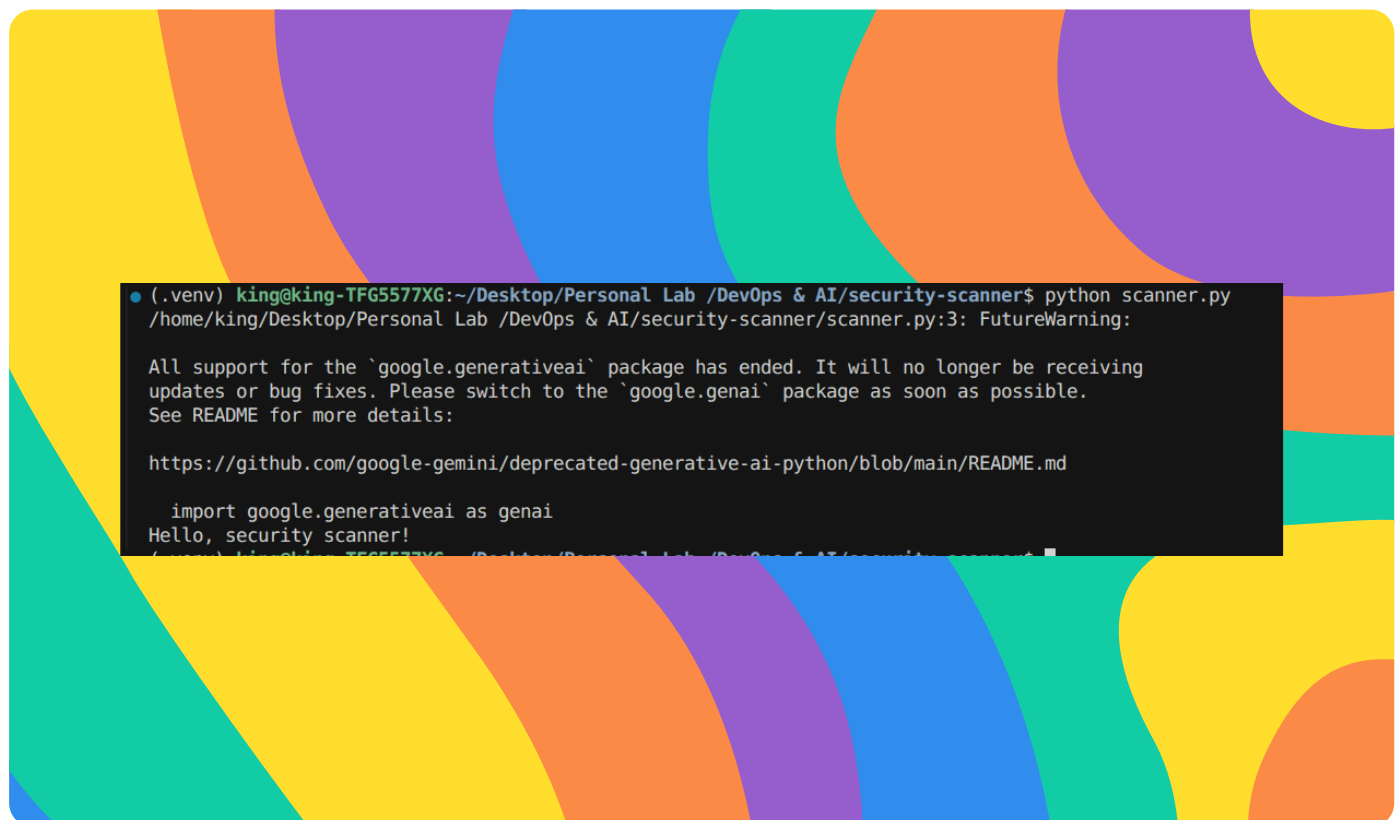
Why I did this project

I did this project today because applications are shipped everyday, hence, it's only right to have a safety net that checks for vulnerabilities in the code. This project met my goals by enlightening me on the ways of software development process.



Connecting to Gemini API

In this step, I'm setting up the Gemini API connection. This involves retrieving the API key from Google and saving it in a ".env file". I need to do this so I can test my connection to Gemini.



```
• (.venv) king@king-TF65577XG:~/Desktop/Personal Lab /DevOps & AI/security-scanner$ python scanner.py
/home/king/Desktop/Personal Lab /DevOps & AI/security-scanner/scanner.py:3: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
Hello, security scanner!
(.venv) king@king-TF65577XG:~/Desktop/Personal Lab /DevOps & AI/security-scanner$
```

I verified the connection by running my test script using "python scanner.py". Gemini responded with "Hello, security scanner!" which confirmed that my prompts have been authenticated and processed by Google's servers.



```
import google.generativeai as genai
The line 'password = "admin123"' itself, while very simple, introduces several critical security vulnerabilities and demonstrates poor security practices. Here's a breakdown:

## Critical Security Issues:

1.  **Hardcoded Password:**
    *   **Issue:** The password `admin123` is directly written into the source code.
    *   **Explanation:** This is perhaps the most significant flaw. Any individual with access to the source code (developers, administrators, version control systems, deployment artifacts, accidental leaks) will immediately know the plaintext password.
    *   **Impact:**
        *   **Insider Threat:** Malicious insiders can easily exploit this.
        *   **Compromise on Code Exposure:** If the code repository becomes public or is breached, the password is leaked.
        *   **Lack of Rotation:** Changing the password requires modifying the code and redeploying, making rotation cumbersome and often neglected.
```

My scanner.py file works by identifying vulnerabilities in test code. When I ran it, Gemini identified the hardcoded credentials and informed me of the risks of exposure. This shows that the Gemini API can be used to analyze to application code for vulnerabilities before pushing to production.



Building the Vulnerability Scanner

In this step, I'm building a vulnerability scanner that detects SQL injection, hardcoded secrets, and weak cryptography - the same vulnerabilities that professional tools look for.

```
Analyzing SQL Injection Example...
=====
Here's the vulnerability analysis:
---
**1. Vulnerability type:** SQL Injection
**Why it's vulnerable:** The 'username' parameter is directly concatenated into the SQL query string without proper sanitization or parameterization.
**Impact:** An attacker can manipulate the query logic, bypass authentication, access sensitive data, or modify/delete database records.
**Secure code fix:** Use parameterized queries, which handle the escaping and quoting of input values automatically.

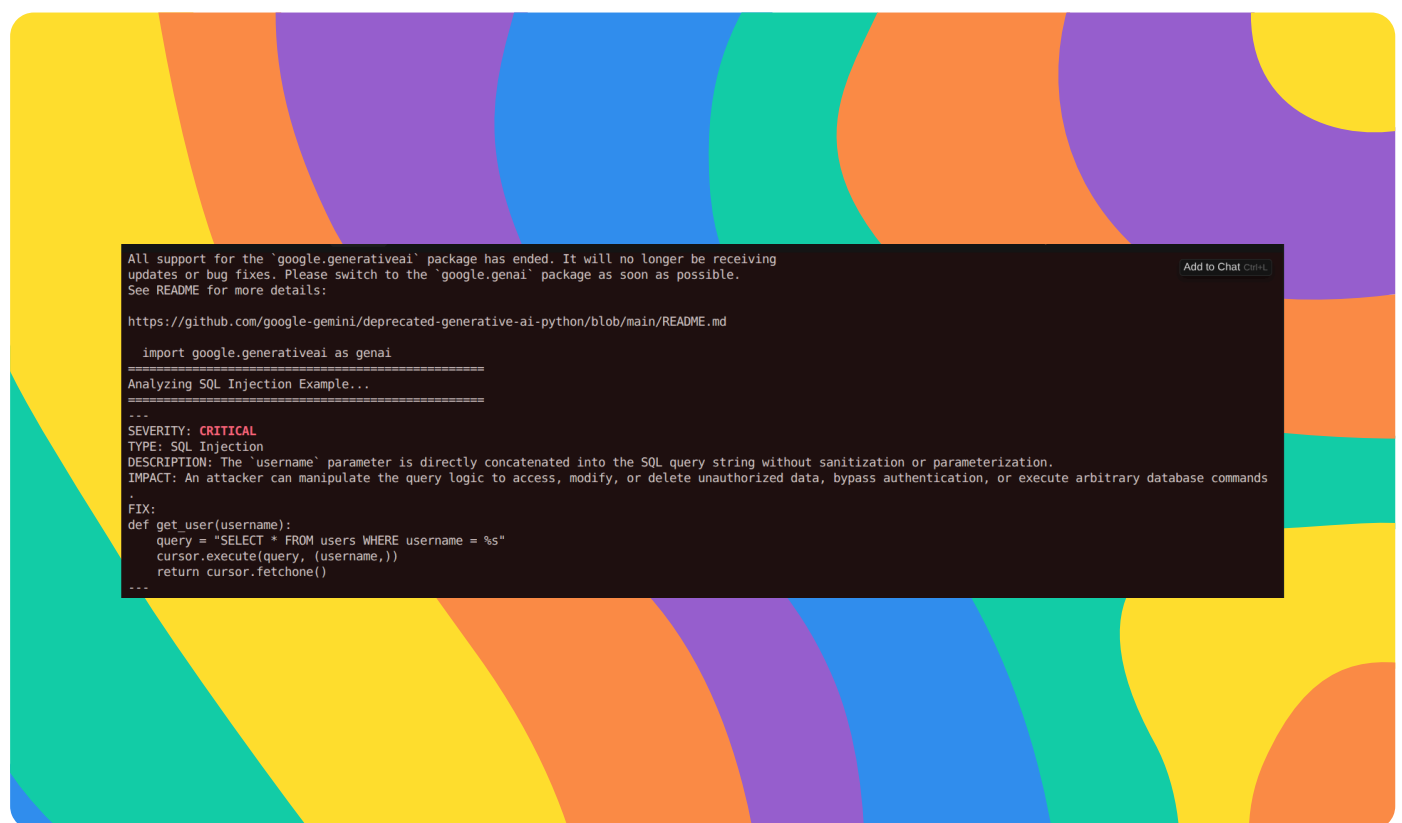
```python
def get_user(username):
 # Use a placeholder (e.g., %s for psycopg2/MySQLdb or ? for sqlite3/pyodbc)
 query = "SELECT * FROM users WHERE username = %s"
 cursor.execute(query, (username,)) # Pass parameters as a tuple/list
 return cursor.fetchone()
```
```

The vulnerabilities Gemini detected were SQL injection, Hardcoded credentials and Weak MD5 hashing(cryptography). The security prompt I crafted asked for the Vulnerability type, why its vulnerable, its impact and a secure code fix . This structured output helps me quickly get a sense of all the vulnerability type within my application and how to fix them.



Adding Severity Ratings

In this step, I'm adding severity ratings which helps me easily prioritize the severity of the vulnerabilities to fix them promptly. I'm also installing colorama to provide me with a colored terminal output for my severity ratings.



I updated the security prompt to include color coded texts in the terminal. The `add_colors_to_output` function works by taking in the text response and color codes them based on the severity ratings. When I see **CRITICAL** in red, it tells me this is a severe vulnerability that must be fixed quickly.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

